Sendra Aptos Bot: Encryption Security & System Architecture

Comprehensive In-Depth Technical Specification for Developers and Security Auditors | Date: August 2026

1. Encryption & Key Management Analysis

The private key security mechanism for the Sendra Aptos Bot is isolated inside aptos-service/src/services/keyManager.ts. Below is the detailed cryptanalysis of how keys are generated, encrypted, stored, and decrypted.

1.1 How ENCRYPTION_KEY in .env is Generated

The application expects a 64-character hexadecimal string representing a raw 32-byte (256-bit) symmetric key loaded via process.env.ENCRYPTION_KEY (or NEW_ENCRYPTION_KEY during wallet rotation):

```
Generation Commands (CSPRNG):
• OpenSSL: openssl rand -hex 32
• Node.js: require('crypto').randomBytes(32).toString('hex')
```

1.2 Cryptographic Algorithm & Strength

- **Algorithm:** aes-256-gcm (Galois/Counter Mode) - Authenticated Encryption with Associated Data (AEAD).
- **Key Size:** 256-bit key length. Highly resistant to brute-force attacks (computationally unbreakable with modern hardware).
- **Authenticity & Integrity (AEAD):** Uses a 16-byte authentication tag (cipher.getAuthTag()). Decryption verifies this tag; any ciphertext or IV modification immediately throws a decryption error.
- **Random Initialization Vector (IV):** Generates a fresh 16-byte random IV (crypto.randomBytes(16)) for every encryption. Prevents ciphertext pattern analysis.
- **Encrypted Output Structure:** Formatted as a colon-delimited string: iv_hex:authTag_hex:ciphertext_hex.

1.3 Vulnerability & Risk Assessment

Risk Vector	Severity	Impact & Technical Description
Plaintext .env Storage	HIGH ■	The master key is stored in plain text in aptos-service/.env. File read vulnerability or server compromise exposes both .env and MySQL database, enabling bulk key decryption.
IV Size Standard Misalignment	MEDIUM ■	keyManager.ts uses 16-byte (128-bit) IVs. NIST SP 800-38D recommends 12-byte (96-bit) IVs for AES-GCM to avoid GHASH preprocessing overhead and potential hash collision risks at extreme scale.
In-Memory RAM Exposure	MEDIUM ■	Decrypted private keys exist as standard V8 JavaScript strings in memory. V8 immutable string garbage collection prevents immediate zeroizing/wiping from process RAM.
Lack of Access Control / Audit	MEDIUM ■	Any request with a valid X-Internal-Secret header can trigger key creation/decryption. No per-user authorization checks or audit log for individual key usage.

1.4 Recommended Hardening Measures

1. **Cloud KMS / Vault:** Replace static .env master key with AWS KMS, GCP KMS, or HashiCorp Vault. Use KMS envelope encryption.
2. **Adjust IV Length:** Update keyManager.ts to crypto.randomBytes(12) (96 bits) per NIST standards.
3. **Trusted Execution Environment:** Deploy aptos-service inside AWS Nitro Enclaves or GCP Confidential VMs.
4. **Enhanced Key Rotation:** Support re-encrypting existing database keys under new KMS key versions without replacing user on-chain Aptos wallet addresses.

2. System Architecture Blueprint

The Sendra Aptos Bot uses a hybrid architecture splitting business logic & user messaging (PHP Core) from cryptographic operations & blockchain execution (Node.js microservice).

```
System Component Map:
1. WhatsApp User ➔ Interacts via WhatsApp chat prompts.
2. PHP Core Backend (MessageController.php) ➔ State machine, PIN authentication, database management.
3. aptos-service (Node.js/Express on :3001) ➔ Key management, Aptos SDK, gasless paymaster, DEX swap.
4. Aptos Indexer Webhook (aptos_indexer_webhook.php) ➔ Listens for on-chain deposit events via GraphQL/Nodit.
```

2.1 Microservice Breakdown (aptos-service)

File Path	Primary Responsibility
src/index.ts	Express application initialization on port 3001, route mounting, global error handling.
src/middleware/internalSecretAuth.ts	Secures endpoints by verifying X-Internal-Secret header against INTERNAL_API_SECRET.
src/services/keyManager.ts	AES-256-GCM encryption/decryption module for Aptos user private keys.
src/services/aptosClient.ts	Instantiates Aptos SDK client (Mainnet/Testnet/Devnet, custom fullnode RPC, Geomi API key).
src/config/aptosAssets.ts	Configures USDC/USDT metadata object addresses, SmoothSend network selection, and explorer URLs.
src/routes/accounts.ts	Routes: POST /accounts/create (generates wallet), POST /accounts/balance (queries APT/USDC/USDT), POST /accounts/validate-key.
src/routes/transactions.ts	Routes: POST /execute-gasless (SmoothSend paymaster), POST /execute (framework transfer), POST /simulate, POST /panora-swap (DEX swap).
src/scripts/rotate-wallets.ts	CLI wallet rotation utility for Sendra users (supports --dry-run and generates JSON audit logs).

3. Transaction Execution & Control Flows

3.1 Gasless Token Transfer Flow (SmoothSend Paymaster)

1. User submits transfer command on WhatsApp -> PHP validates user PIN -> PHP calls POST /transactions/execute-gasless on aptos-service.
2. aptos-service calls keyManager.decryptKey() to obtain raw private key in memory.
3. Builds transaction payload via Aptos SDK and signs with sender authenticator.
4. Submits payload to **SmoothSend Relayer** (sponsor pays APT gas fees).
5. **Chain Recovery Safeguard:** If SmoothSend API times out, submitGaslessWithChainRecovery() polls the Aptos ledger for sequence number to verify if transaction landed on-chain.
6. Returns transaction hash and explorer URL to PHP -> WhatsApp confirmation sent to user.

3.2 Deposit Detection & Automated Notification Flow

1. External sender transfers USDC/USDT to user's Sendra Aptos address.
2. aptos_indexer_webhook.php polls Aptos Indexer GraphQL / Nodit for FungibleAsset::DepositEvent.
3. Matches receiving address against MySQL users table.
4. Updates user balance tracking and triggers WhatsApp alert: *'You received X USDC'*.